

Experiment ‘p_3_3_number_of_digits’ Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_3_3_number_of_digits

Natural language statement: Write a method that takes an integer and returns how many digits the number has, by checking whether the number is 10, 100, and so on. (Assume that all integers are less than ten billion.) If the number is negative, first multiply it with -1 .

Method signature: p_3_3_number_of_digits (i: int) returns (n: nat)

Ensures

- $i \geq 0 \implies n == \text{digits}(i)$
- $i < 0 \implies n == \text{digits}(-i)$

Requires

- $10000000000 > i > -10000000000$

Functional Code Given

```
function digits (n: nat): nat
{
  if 0 <= n <= 9 then 1
  else 1 + digits (n / 10)
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes an integer and returns how many digits the number has, by
→ checking whether the number is 10, 100, and so on. (Assume that all integers are
→ less than ten billion.) If the number is negative, first multiply it with -1.

The signature should be:

`p_3_3_number_of_digits (i: int) returns (n: nat)`

The method should respect the following contract:

`requires 10000000000 > i > -10000000000, ensures i >= 0 ==> n == digits(i), ensures i < 0`
→ `==> n == digits(-i)`

The contract uses the following dafny code:

```
function digits (n: nat): nat
{
  if 0 <= n <= 9 then 1
  else 1 + digits (n / 10)
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dmdjpai0BG6Llant3OpgIediJzhJ

Model: gpt-5.5-2026-04-23

Created at: 1780484869

Finish reason: stop

Usage: 528 tokens in, and 2967 tokens out

```
{
  var x: int := i;
  if x < 0 {
    x := -x;
  }

  assert 0 <= x;
  assert x < 10000000000;
```

```

if x >= 1000000000 {
  assert 1000000000 <= x < 10000000000;
  n := 10;
} else if x >= 100000000 {
  assert 100000000 <= x < 1000000000;
  n := 9;
} else if x >= 10000000 {
  assert 10000000 <= x < 100000000;
  n := 8;
} else if x >= 1000000 {
  assert 1000000 <= x < 10000000;
  n := 7;
} else if x >= 100000 {
  assert 100000 <= x < 1000000;
  n := 6;
} else if x >= 10000 {
  assert 10000 <= x < 100000;
  n := 5;
} else if x >= 1000 {
  assert 1000 <= x < 10000;
  n := 4;
} else if x >= 100 {
  assert 100 <= x < 1000;
  n := 3;
} else if x >= 10 {
  assert 10 <= x < 100;
  n := 2;
} else {
  assert 0 <= x < 10;
  n := 1;
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_3_3_number_of_digits/program.dfy(11,0): Error: a postcondition could not
  ↪ be proved on this return path

```

```

|
11 | {
| ^

```

```

problem_data/p_3_3_number_of_digits/program.dfy(9,22): Related location: this is the
  ↪ postcondition that could not be proved

```

```

|
9 | ensures i >= 0 ==> n == digits(i)
| ^^

```

```
problem_data/p_3_3_number_of_digits/program.dfy(11,0): Error: a postcondition could not
  ↳ be proved on this return path
```

```
|
11 | {
| ~
```

```
problem_data/p_3_3_number_of_digits/program.dfy(10,21): Related location: this is the
  ↳ postcondition that could not be proved
```

```
|
10 | ensures i < 0 ==> n == digits(-i)
| ^^
```

Dafny program verifier finished with 2 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
↳ Dafny body, including the curly braces that surround it. Do not show the signature
↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dmdl3Iwvzq9F7vxfyELT34n26dSgg

Model: gpt-5.5-2026-04-23

Created at: 1780484945

Finish reason: stop

Usage: 1218 tokens in, and 6025 tokens out

```
{
  var x: int;
  if i < 0 {
    x := -i;
  } else {
    x := i;
  }

  assert 0 <= x;
  assert x < 10000000000;
  var y: nat := x;

  var a0: nat := y;
  var a1: nat := a0 / 10;
  var a2: nat := a1 / 10;
  var a3: nat := a2 / 10;
  var a4: nat := a3 / 10;
  var a5: nat := a4 / 10;
  var a6: nat := a5 / 10;
  var a7: nat := a6 / 10;
  var a8: nat := a7 / 10;
  var a9: nat := a8 / 10;
```

```

if y >= 1000000000 {
  n := 10;
  assert 100000000 <= a1 < 1000000000;
  assert 10000000 <= a2 < 100000000;
  assert 1000000 <= a3 < 10000000;
  assert 100000 <= a4 < 1000000;
  assert 10000 <= a5 < 100000;
  assert 1000 <= a6 < 10000;
  assert 100 <= a7 < 1000;
  assert 10 <= a8 < 100;
  assert 0 <= a9 <= 9;
  assert digits(a9) == 1;
  assert digits(a8) == 1 + digits(a9);
  assert digits(a8) == 2;
  assert digits(a7) == 1 + digits(a8);
  assert digits(a7) == 3;
  assert digits(a6) == 1 + digits(a7);
  assert digits(a6) == 4;
  assert digits(a5) == 1 + digits(a6);
  assert digits(a5) == 5;
  assert digits(a4) == 1 + digits(a5);
  assert digits(a4) == 6;
  assert digits(a3) == 1 + digits(a4);
  assert digits(a3) == 7;
  assert digits(a2) == 1 + digits(a3);
  assert digits(a2) == 8;
  assert digits(a1) == 1 + digits(a2);
  assert digits(a1) == 9;
  assert digits(a0) == 1 + digits(a1);
  assert digits(a0) == 10;
  assert n == digits(a0);
} else if y >= 100000000 {
  n := 9;
  assert 10000000 <= a1 < 100000000;
  assert 1000000 <= a2 < 10000000;
  assert 100000 <= a3 < 1000000;
  assert 10000 <= a4 < 100000;
  assert 1000 <= a5 < 10000;
  assert 100 <= a6 < 1000;
  assert 10 <= a7 < 100;
  assert 0 <= a8 <= 9;
  assert digits(a8) == 1;
  assert digits(a7) == 1 + digits(a8);
  assert digits(a7) == 2;
  assert digits(a6) == 1 + digits(a7);
  assert digits(a6) == 3;
  assert digits(a5) == 1 + digits(a6);
  assert digits(a5) == 4;
  assert digits(a4) == 1 + digits(a5);
  assert digits(a4) == 5;
  assert digits(a3) == 1 + digits(a4);

```

```

    assert digits(a3) == 6;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 7;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 8;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 9;
    assert n == digits(a0);
} else if y >= 10000000 {
    n := 8;
    assert 1000000 <= a1 < 10000000;
    assert 100000 <= a2 < 1000000;
    assert 10000 <= a3 < 100000;
    assert 1000 <= a4 < 10000;
    assert 100 <= a5 < 1000;
    assert 10 <= a6 < 100;
    assert 0 <= a7 <= 9;
    assert digits(a7) == 1;
    assert digits(a6) == 1 + digits(a7);
    assert digits(a6) == 2;
    assert digits(a5) == 1 + digits(a6);
    assert digits(a5) == 3;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 4;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 5;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 6;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 7;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 8;
    assert n == digits(a0);
} else if y >= 1000000 {
    n := 7;
    assert 100000 <= a1 < 1000000;
    assert 10000 <= a2 < 100000;
    assert 1000 <= a3 < 10000;
    assert 100 <= a4 < 1000;
    assert 10 <= a5 < 100;
    assert 0 <= a6 <= 9;
    assert digits(a6) == 1;
    assert digits(a5) == 1 + digits(a6);
    assert digits(a5) == 2;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 3;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 4;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 5;
    assert digits(a1) == 1 + digits(a2);

```

```

    assert digits(a1) == 6;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 7;
    assert n == digits(a0);
} else if y >= 100000 {
    n := 6;
    assert 10000 <= a1 < 100000;
    assert 1000 <= a2 < 10000;
    assert 100 <= a3 < 1000;
    assert 10 <= a4 < 100;
    assert 0 <= a5 <= 9;
    assert digits(a5) == 1;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 2;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 3;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 4;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 5;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 6;
    assert n == digits(a0);
} else if y >= 10000 {
    n := 5;
    assert 1000 <= a1 < 10000;
    assert 100 <= a2 < 1000;
    assert 10 <= a3 < 100;
    assert 0 <= a4 <= 9;
    assert digits(a4) == 1;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 2;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 3;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 4;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 5;
    assert n == digits(a0);
} else if y >= 1000 {
    n := 4;
    assert 100 <= a1 < 1000;
    assert 10 <= a2 < 100;
    assert 0 <= a3 <= 9;
    assert digits(a3) == 1;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 2;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 3;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 4;

```

```

    assert n == digits(a0);
} else if y >= 100 {
    n := 3;
    assert 10 <= a1 < 100;
    assert 0 <= a2 <= 9;
    assert digits(a2) == 1;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 2;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 3;
    assert n == digits(a0);
} else if y >= 10 {
    n := 2;
    assert 0 <= a1 <= 9;
    assert digits(a1) == 1;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 2;
    assert n == digits(a0);
} else {
    n := 1;
    assert 0 <= a0 <= 9;
    assert digits(a0) == 1;
    assert n == digits(a0);
}

assert a0 == y;
assert n == digits(y);
if i < 0 {
    assert x == -i;
    assert y == -i;
    assert n == digits(-i);
} else {
    assert x == i;
    assert y == i;
    assert n == digits(i);
}
}

```

Final Program

```

function digits (n: nat): nat
{
    if 0 <= n <= 9 then 1
    else 1 + digits (n / 10)
}

method p_3_3_number_of_digits (i: int) returns (n: nat)
requires 100000000000 > i > -100000000000
ensures i >= 0 ==> n == digits(i)
ensures i < 0 ==> n == digits(-i)

```



```

{
  var x: int;
  if i < 0 {
    x := -i;
  } else {
    x := i;
  }

  assert 0 <= x;
  assert x < 10000000000;
  var y: nat := x;

  var a0: nat := y;
  var a1: nat := a0 / 10;
  var a2: nat := a1 / 10;
  var a3: nat := a2 / 10;
  var a4: nat := a3 / 10;
  var a5: nat := a4 / 10;
  var a6: nat := a5 / 10;
  var a7: nat := a6 / 10;
  var a8: nat := a7 / 10;
  var a9: nat := a8 / 10;

  if y >= 1000000000 {
    n := 10;
    assert 100000000 <= a1 < 1000000000;
    assert 10000000 <= a2 < 100000000;
    assert 1000000 <= a3 < 10000000;
    assert 100000 <= a4 < 1000000;
    assert 10000 <= a5 < 100000;
    assert 1000 <= a6 < 10000;
    assert 100 <= a7 < 1000;
    assert 10 <= a8 < 100;
    assert 0 <= a9 <= 9;
    assert digits(a9) == 1;
    assert digits(a8) == 1 + digits(a9);
    assert digits(a8) == 2;
    assert digits(a7) == 1 + digits(a8);
    assert digits(a7) == 3;
    assert digits(a6) == 1 + digits(a7);
    assert digits(a6) == 4;
    assert digits(a5) == 1 + digits(a6);
    assert digits(a5) == 5;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 6;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 7;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 8;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 9;
  }
}

```

```

    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 10;
    assert n == digits(a0);
} else if y >= 100000000 {
    n := 9;
    assert 10000000 <= a1 < 100000000;
    assert 1000000 <= a2 < 10000000;
    assert 100000 <= a3 < 1000000;
    assert 10000 <= a4 < 100000;
    assert 1000 <= a5 < 10000;
    assert 100 <= a6 < 1000;
    assert 10 <= a7 < 100;
    assert 0 <= a8 <= 9;
    assert digits(a8) == 1;
    assert digits(a7) == 1 + digits(a8);
    assert digits(a7) == 2;
    assert digits(a6) == 1 + digits(a7);
    assert digits(a6) == 3;
    assert digits(a5) == 1 + digits(a6);
    assert digits(a5) == 4;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 5;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 6;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 7;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 8;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 9;
    assert n == digits(a0);
} else if y >= 10000000 {
    n := 8;
    assert 1000000 <= a1 < 10000000;
    assert 100000 <= a2 < 1000000;
    assert 10000 <= a3 < 100000;
    assert 1000 <= a4 < 10000;
    assert 100 <= a5 < 1000;
    assert 10 <= a6 < 100;
    assert 0 <= a7 <= 9;
    assert digits(a7) == 1;
    assert digits(a6) == 1 + digits(a7);
    assert digits(a6) == 2;
    assert digits(a5) == 1 + digits(a6);
    assert digits(a5) == 3;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 4;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 5;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 6;

```

```

    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 7;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 8;
    assert n == digits(a0);
} else if y >= 1000000 {
    n := 7;
    assert 100000 <= a1 < 1000000;
    assert 10000 <= a2 < 100000;
    assert 1000 <= a3 < 10000;
    assert 100 <= a4 < 1000;
    assert 10 <= a5 < 100;
    assert 0 <= a6 <= 9;
    assert digits(a6) == 1;
    assert digits(a5) == 1 + digits(a6);
    assert digits(a5) == 2;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 3;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 4;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 5;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 6;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 7;
    assert n == digits(a0);
} else if y >= 100000 {
    n := 6;
    assert 10000 <= a1 < 100000;
    assert 1000 <= a2 < 10000;
    assert 100 <= a3 < 1000;
    assert 10 <= a4 < 100;
    assert 0 <= a5 <= 9;
    assert digits(a5) == 1;
    assert digits(a4) == 1 + digits(a5);
    assert digits(a4) == 2;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 3;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 4;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 5;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 6;
    assert n == digits(a0);
} else if y >= 10000 {
    n := 5;
    assert 1000 <= a1 < 10000;
    assert 100 <= a2 < 1000;
    assert 10 <= a3 < 100;

```

```

    assert 0 <= a4 <= 9;
    assert digits(a4) == 1;
    assert digits(a3) == 1 + digits(a4);
    assert digits(a3) == 2;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 3;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 4;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 5;
    assert n == digits(a0);
} else if y >= 1000 {
    n := 4;
    assert 100 <= a1 < 1000;
    assert 10 <= a2 < 100;
    assert 0 <= a3 <= 9;
    assert digits(a3) == 1;
    assert digits(a2) == 1 + digits(a3);
    assert digits(a2) == 2;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 3;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 4;
    assert n == digits(a0);
} else if y >= 100 {
    n := 3;
    assert 10 <= a1 < 100;
    assert 0 <= a2 <= 9;
    assert digits(a2) == 1;
    assert digits(a1) == 1 + digits(a2);
    assert digits(a1) == 2;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 3;
    assert n == digits(a0);
} else if y >= 10 {
    n := 2;
    assert 0 <= a1 <= 9;
    assert digits(a1) == 1;
    assert digits(a0) == 1 + digits(a1);
    assert digits(a0) == 2;
    assert n == digits(a0);
} else {
    n := 1;
    assert 0 <= a0 <= 9;
    assert digits(a0) == 1;
    assert n == digits(a0);
}

assert a0 == y;
assert n == digits(y);
if i < 0 {

```

```
    assert x == -i;
    assert y == -i;
    assert n == digits(-i);
} else {
    assert x == i;
    assert y == i;
    assert n == digits(i);
}
}
```

Total Token Usage

Input tokens: 1746

Output tokens: 8992

Reasoning tokens: 6080

Sum of ‘total tokens’: 10738

Experiment Timings

Overall Experiment started at 1780484869015, ended at 1780485039151, lasting 170136ms (170.14 seconds)

Iteration #1 started at 1780484869017, ended at 1780484945156, lasting 76139ms (76.14 seconds)

Iteration #2 started at 1780484945157, ended at 1780485039151, lasting 93994ms (93.99 seconds)